

AMD-Agent: Autonomous Malware Detection Agent

Baruh Ifraimov¹ and Dor Cohen¹ Department of Cyber Security
Ariel University Center of Samaria, Ariel, Israel

Abstract—This project presents AMD-Agent, an autonomous LLM-assisted continual-learning system for PE malware detection. The agent collects malware and benign samples from live sources, validates downloaded binaries, extracts static PE features, monitors concept drift, and retrain a LightGBM classifier using replay-based learning. A local Ollama-based LLM agent supports the workflow by assisting with source selection, CTI filtering, PE error triage, and drift explanations, while deterministic fallbacks keep the system operational when the LLM is unavailable. To evaluate updates reliably, AMD-Agent compares the previous and updated models on a strict temporal holdout excluded from training, reporting Accuracy, Precision, Recall, and False Positive Rate (FPR). The result is an adaptive malware detection pipeline that combines autonomous collection, LLM-assisted decision making, continual retraining, and measurable before/after evaluation. Our implementation and source code are available on GitHub [1].

I. INTRODUCTION

Malware detection is a continuously changing problem. New malware variants, packers, and evasion techniques appear over time, while benign software also changes through new versions and build processes. A classifier trained once on a fixed dataset may therefore become outdated when deployed in a live environment. This motivates the need for continual learning and reliable update evaluation.

AMD-Agent addresses this problem by implementing an autonomous pipeline for PE malware detection without relying on predefined datasets. The system discovers malware and benign samples from external sources, downloads and validates PE files, extracts static features, monitors drift, performs inference, and retrains the model when new labeled data or drift evidence is available. The classifier is based on LightGBM with feature selection and FPR-aware threshold calibration.

A key part of the project is avoiding misleading evaluation. For each model update, AMD-Agent creates a strict chronological holdout and excludes it from retraining. The previous and updated models are then evaluated on the same holdout using Accuracy, Precision, Recall, and FPR. This enables a direct before/after comparison while reducing data leakage.

The system also includes optional local LLM support through Ollama for source selection, CTI filtering, and drift explanations. The LLM is not the malware classifier; it assists the agent’s workflow while deterministic fallbacks keep the system operational. Overall, AMD-Agent demonstrates a practical approach for adaptive malware detection with automated collection, retraining, and evaluation.

II. RELATED WORK

This section reviews the fundamental literature and tooling that influenced the architecture of AMD-Agent. A consolidated critical analysis of their methodologies, limitations, and direct contributions to our system design is provided in Table I.

Data Acquisition and Parsing: To facilitate an autonomous pipeline without predefined datasets, we utilize the **MalwareBazaar API** [2] as our primary live malware source. While it provides metadata-rich, real-world samples, it requires strict sandboxing to mitigate execution risks. To safely process these incoming streams, we rely on *pefile* [3], a mature Python library for static Windows Portable Executable (PE) parsing. Although limited by its inability to observe runtime behavior, its safe API perfectly aligns with our lightweight feature extraction stage.

Feature Representation and Continual Learning: Our structural feature vector is directly inspired by **EMBER** [4], which establishes a strong baseline for static malware classification, despite its inherent vulnerability to advanced packing techniques. Because these static distributions shift over time, we employ **River’s ADWIN** algorithm [5] for lightweight, online drift detection based on section entropy. Once drift is identified, we address catastrophic forgetting by adopting the diversity-aware replay methodology proposed in **MADAR** [6], mixing historical samples with new data during **LightGBM** retraining. **Optuna** [7] is integrated into this retraining phase to automate hyperparameter optimization via tree-structured parzen estimators, ensuring that the updated LightGBM model configuration adapts efficiently to the shifted feature space without manual tuning.

Evaluation Methodology: Finally, to evaluate our model updates reliably, we adopted the **TESSERACT** [8] evaluation framework. Random train-test splits often overestimate malware classification performance due to temporal data leakage. By enforcing strict chronological splits, **TESSERACT** ensures a realistic before/after comparison, even though it occasionally requires skipping evaluation phases when temporal windows yield single-class holdouts.

III. SYSTEM ARCHITECTURE AND AGENT DESIGN

AMD-Agent operates as a graph-based controller managed by a LangGraph state machine. Rather than executing as a single monolithic script, the system can run as a persistent loop traversing distinct collection, ML, and evaluation forks.

TABLE I
SUMMARY OF PAPERS AND TOOLS USED IN AMD-AGENT

Item	Type	Purpose	Pros / Cons	How We Used It
TESSERACT [8]	Paper / Evaluation	Chronological malware evaluation.	Pros: reduces temporal leakage. Cons: can skip when splits are single-class.	Used for time-aware Accuracy, Precision, Recall, and FPR evaluation, including before/after model-update comparison.
EMBER [4]	Paper / Method	Static PE malware feature representation.	Pros: strong malware ML baseline. Cons: static features can be evaded by packing.	Inspired our 2304-dimensional EMBER-like static feature vector.
MADAR [6]	Paper / Method	Replay-based continual learning.	Pros: reduces catastrophic forgetting. Cons: depends on stored historical samples.	Guided family-aware replay selection before retraining with new drifted samples.
LangGraph [9]	Tool	Agent workflow orchestration.	Pros: modular routing and state passing. Cons: requires careful state cleanup.	Built the graph nodes for source selection, fetch, validation, feature extraction, drift detection, retraining, inference, and evaluation.
pefile [3]	Tool	Static parsing of Windows PE files.	Pros: mature and safe. Cons: static-only, no runtime behavior.	Extracted PE headers, sections, directories, imports, exports, entropy, and parser warnings.
Ollama [10]	Tool	Local LLM semantic sidecar.	Pros: local semantic analysis. Cons: best-effort output, requires fallback logic.	Used for source strategy support, CTI hash verdicts, PE parse-error triage, optional source-page refinement, and drift explanations.
Optuna [7]	Tool	Automated hyperparameter optimization.	Pros: efficient Bayesian search (TPE), time-capped trials. Cons: high search overhead if unconstrained.	Ran up to 25 time-capped trials on the 384-feature validation slice to optimize LightGBM hyperparameters under strict FPR constraints.
MalwareBazaar [2]	Tool / API	Live malware hash and sample source.	Pros: real-world malware and metadata. Cons: quotas, noisy samples, availability limits.	Used for malware discovery, PE hash verification, and controlled sample download.
OTX [11]	Tool / API	Live CTI pulse source.	Pros: fresh threat intelligence. Cons: noisy and unstructured text.	Extracted SHA256 indicators from pulses, filtered context with Ollama, then validated hashes before queueing.
River / ADWIN [5]	Tool / Algorithm	Streaming concept drift detection.	Pros: online and lightweight. Cons: threshold tuning is required.	Monitored entropy shifts and helped trigger retraining when drift appeared.

A. State Management

The system separates short-term runtime data from long-term stored data. This keeps each pipeline run simple while still allowing the agent to continue learning over time.

- 1) **Current Run State (AgentState):** Each pipeline run begins with a newly initialized `AgentState`. This object retains only the data required for the active batch, including discovered hashes, downloaded file paths, extracted feature vectors, predictions, and routing flags (e.g., `drift_detected`). To guarantee isolation, the source selector clears these fields before each run so legacy data cannot affect the new batch.
- 2) **LangGraph Execution State:** LangGraph uses this state while moving between nodes such as source selection, download, validation, feature extraction, drift monitoring, inference, retraining, and evaluation. The `MemorySaver` checkpointer helps LangGraph manage the state during execution, but it is not treated as the system’s permanent memory.
- 3) **Persistent Storage:** Long-term information is stored outside the graph state. SQLite stores the sample database, including hashes, file paths, labels, extracted features, predictions, sample status, and ingestion time. Corrupted samples are marked in the database so they are not repeatedly processed. The trained LightGBM model is saved with Joblib as `model.pkl`, and the ADWIN drift detector state is saved as `adwin.joblib`. Evaluation, drift, and model update records are written to log files under `data/`.

B. Dynamic Feedback Control Loop

The architecture relies on a closed-loop feedback mechanism governed by three main routing decisions (Algorithm 1):

- **Fork A (Collection):** Evaluates SQLite class distributions and provider yield metrics recorded in SQLite. If the system lacks specific classes, it routes to discovery modules targeting those deficits (e.g., Sysinternals for benign files). Sources are ranked by recent success rate, low-yield providers can be cooled down, and collection/fetch/validation/update steps use reusable tools.
- **Fork B (Machine Learning):** Evaluates the current batch against the ADWIN monitor. If drift is flagged, the graph routes dynamically to the continual learning (MADAR) nodes; otherwise, it passes directly to the inference node.
- **Fork C (Evaluation Gate):** Triggers chronological TESSERACT evaluations periodically every N steady runs and after drift or retraining. After each successful model update, the previous and updated models are compared on the same strict temporal holdout, and the delta is written to `model_update_log.jsonl`.

C. LLM Sidecar Intelligence (Ollama)

Ollama is used as a local semantic sidecar, not as the malware classifier and not as a safety gate. It supports the agent in several roles: source strategy selection with reusable CTI tools, structured filtering of unstructured CTI text from OTX and curated feeds into SHA256 malware verdicts, PE

Algorithm 1 AMD-Agent Core Control Loop

Require: SQLite DB $\mathcal{C}$, Classifier $\mathcal{M}$, ADWIN $\mathcal{A}$

```
1: loop
2:    $L \leftarrow \text{BalanceController}(\mathcal{C}.\text{counts})$  {Determine needed class}

3:    $B_{\text{batch}} \leftarrow \text{DownloadAndValidate}(L)$ 
4:   for each file  $f \in B_{\text{batch}}$  do
5:      $\mathbf{x}_f \leftarrow \text{ExtractFeatures}(f)$ 
6:      $\mathcal{C}.\text{Insert}(f, \mathbf{x}_f)$ 
7:   end for
8:   if  $\text{Drift}(\mathcal{A}, \mathbf{x}_B, \text{Entropy}_B)$  then
9:      $\text{Trigger} \leftarrow \text{Drift}$ 
10:  else if  $\mathcal{C}.\text{UntrainedCount} \geq 50$  then
11:     $\text{Trigger} \leftarrow \text{Threshold}$ 
12:  else
13:     $\text{Trigger} \leftarrow \text{None}$ 
14:  end if
15:  if  $\text{Trigger} \in \{\text{Drift}, \text{Threshold}\}$  then
16:     $\mathcal{M} \leftarrow \text{MadarRetrain}(\mathcal{C}, B_{\text{batch}})$ 
17:  else
18:    for each file  $f \in B_{\text{batch}}$  do
19:       $\mathcal{C}_f.\text{prediction} \leftarrow \text{Predict}(\mathcal{M}, \mathbf{x}_f)$ 
20:    end for
21:  end if
22:  if  $\text{EvalDue}$  or  $\text{Trigger} \neq \text{None}$  then
23:     $\text{LogTesseractEval}(\mathcal{C}, \mathcal{M})$ 
24:  end if
25: end loop
```

parse-error triage during feature extraction, optional PE source page refinement, and explanation of concept-drift events from drift statistics and anomalous PE features.

All LLM outputs are best-effort and have deterministic fallbacks. Candidate samples still pass through hard validation gates: MalwareBazaar PE verification, MZ and PE\0\0 signature checks, SHA256 filename verification, duplicate detection, and SQLite quarantine for corrupted samples. This design provides LLM-based content analysis, autonomous source reasoning, and drift explainability without making pipeline correctness depend on the LLM.

IV. METHODOLOGY

A. Safe Malware Acquisition and Sandboxing

Because the system continuously ingests live, weaponized PE files, the pipeline enforces strict isolation controls. It runs within a dedicated Docker bridge network (`malware_net`), while `iptables` egress rules block outbound traffic to private subnets, preserving required DNS, Ollama, and public access.

Crucially, **downloaded binaries are never executed**. Files are written directly to a sandbox directory with restrictive `0600` permissions. Before progressing to feature extraction, deterministic validation gates verify the DOS MZ header, the PE\0\0 signature at `elfanew`, the SHA256 filename/content match, and duplicate status. Files failing these checks are logged to SQLite as `corrupted` and permanently quarantined to prevent cyclical re-fetching.

B. Static Feature Representation

Static analysis projects each validated binary into a fixed-size feature space without requiring execution. Each file maps to a uniform vector:

$$\mathbf{x} = (f_1, f_2, \dots, f_{2304}) \in \mathbb{R}^{2304} \quad (1)$$

This 2304-dimensional EMBER-like representation combines PE headers and directories, section metadata, import/export information, printable string statistics, byte histograms, byte-entropy histograms, Authenticode and overlay indicators, and lightweight instruction-level features:

- **Entropy and Packing Signals:** Section entropy and byte-window entropy variations, which are sensitive to packing, encryption, and obfuscation, are calculated as:

$$H(X) = - \sum_{b=0}^{255} p(b) \log_2 p(b) \quad (2)$$

where $p(b)$ is the byte frequency.

- **PE Structure and Behavioral Surface:** The extractor records structural and behavioral static signals such as section permissions, suspicious section names, imported APIs, exported symbols, URLs, filesystem paths, registry strings, executable-memory APIs, and lightweight instruction-level patterns. This preserves a fully static pipeline while capturing indicators related to packing, persistence, API usage, and code behavior.

C. Classifier and Dynamic Threshold Calibration

To optimize classification speed, an XGBoost-based feature ranking step evaluates the $\mathbb{R}^{2304}$ matrix and isolates the top 384 dimensions by feature importance. A LightGBM binary classifier is then trained exclusively on this subset.

In cybersecurity operations, False Positive Rate (FPR) containment is paramount. After hyperparameter tuning via Optuna (a Python library for automatic hyperparameter search), the classification threshold τ is dynamically calibrated on the validation split. The target FPR becomes strictly tighter as the number of trainable benign samples (N_0) in the corpus grows:

$$\text{target_fpr}(N_0) = \begin{cases} 0.05 & N_0 < 1000 \\ 0.01 & 1000 \leq N_0 < 5000 \\ 0.001 & N_0 \geq 5000 \end{cases} \quad (3)$$

The FPR target starts relaxed during bootstrap, then tightens automatically to ($\leq 0.1\%$) once the benign corpus is large enough for steady-state operation.

D. Drift Detection

The agent checks for concept drift (when malware patterns change) using two methods. If either check triggers, the graph routes the current pass to explanation and retraining instead of normal inference.

Entropy stream (ADWIN). For each validated file, the system computes the average PE section entropy and streams it into River’s ADWIN detector, which raises an alert if the entropy distribution suddenly shifts.

Feature stream (sliding windows). The system also tracks its top $d = 64$ features. It compares the most recent $W = 20$ samples (the current window) against the 20 samples right before them (the previous window). It looks for two types of changes:

1. **Mean shift:** Did the average values of the features move?

$$\text{mean_shift} = \frac{1}{d} \sum_{j=1}^d \frac{|\mu_j^{\text{curr}} - \mu_j^{\text{prev}}|}{\sigma_j^{\text{prev}} + \epsilon} \quad (4)$$

2. **Correlation shift:** Do the features still relate to each other in the same way? This checks the relationships between dimensions, not just their individual levels. It calculates the mean absolute difference between the Pearson correlation matrices (C^{prev} and C^{curr}) of the two windows:

$$\Delta_{\text{corr}} = \text{mean}(|C^{\text{curr}} - C^{\text{prev}}|) \quad (5)$$

Trigger Conditions. The system officially flags a drift if any of these limits are crossed:

$$\text{drift} \Leftrightarrow \text{ADWIN}_{\text{alert}} \vee \text{mean_shift} \geq 1.5 \vee \Delta_{\text{corr}} \geq 0.35 \quad (6)$$

Autonomous Mitigation. When drift happens, the agent saves its current performance scores. An LLM can optionally write a plain-text summary explaining what changed. Next, the model retrains itself (the MADAR method). It mixes the new batch of files with up to 3,000 carefully chosen past files so it does not forget older malware. To ensure the new model is tested fairly, the newest 15% of all data is kept completely hidden during training.

E. Catastrophic Forgetting Mitigation (MADAR Replay)

To avoid forgetting older malware and benign patterns during retraining, the system uses a MADAR-inspired replay strategy. When drift or a retraining threshold triggers an update, the current drift batch is combined with a bounded replay sample from historical SQLite records, while the strict temporal holdout is excluded from training to keep evaluation clean.

The replay budget is capped at $\text{REPLAY_BUDGET} = 3000$. Half of the budget is reserved for benign samples selected by uniform random sampling. The remaining half is assigned to malware families using proportional ratio budgeting, so larger observed families receive more replay slots while smaller families are still represented. Inside each malware family, an Isolation Forest ranks samples by anomaly score; the replay set keeps both representative low-anomaly samples and high-anomaly edge cases. This gives the retraining step a compact memory of past decision boundaries while still adapting to newly observed drifted samples.

V. EXPERIMENTAL RESULTS AND ANALYSIS

The following sections detail the empirical evaluation of the AMD-Agent pipeline, using chronological TESSERACT-style evaluation and model-update logs to reduce temporal leakage.

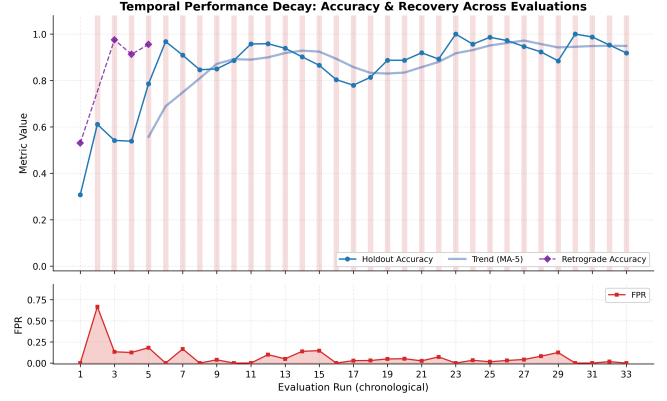


Fig. 1. Accuracy & Recovery Across Evaluation

A. Temporal Performance Decay Analysis

The chronological evaluation shows the expected early instability of a live-learning system, followed by recovery as additional batches are collected and retraining cycles accumulate. Holdout accuracy rises from weak early runs to mostly high steady-state values, while FPR becomes substantially lower after the initial spikes. The moving trend and retrograde checks show that the agent is not only fitting the newest batch, but also preserving useful historical behavior across later evaluation windows.

B. Error and Threshold Analysis

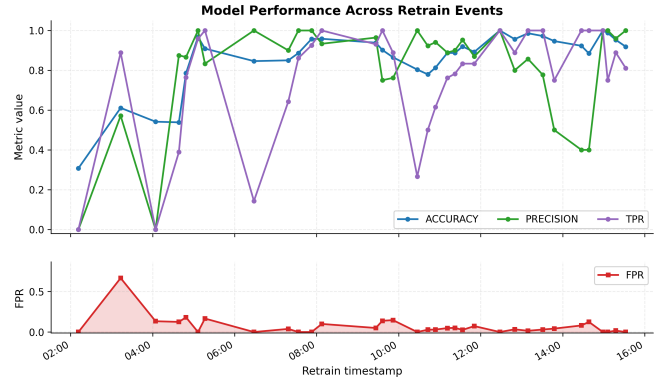


Fig. 2. Accuracy, precision, TPR, and FPR across retrain events.

The model is calibrated around operational FPR control rather than accuracy alone. The results show that early runs can produce unstable FPR, but later retraining generally keeps FPR low while accuracy, precision, and TPR remain high. False negatives may occur when packed or obfuscated samples hide static indicators, so the system records retraining deltas and threshold behavior instead of assuming one fixed model remains reliable over time.

C. Data Provenance and Source Reliability

The final active corpus is drawn from multiple providers rather than a predefined dataset: `benign_net` contributes 225 samples, MalwareBazaar 199, Sysinternals 99, ThreatFox

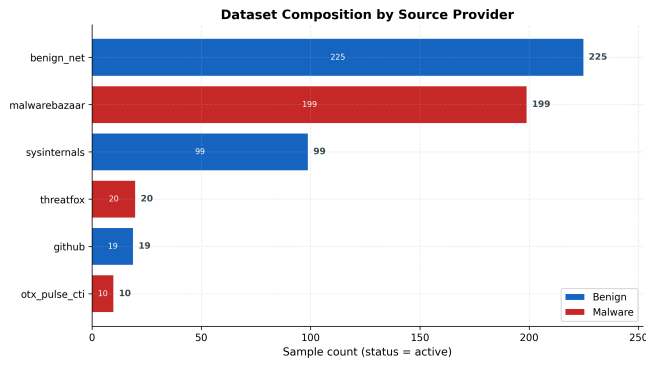


Fig. 3. Final active dataset composition by source provider.

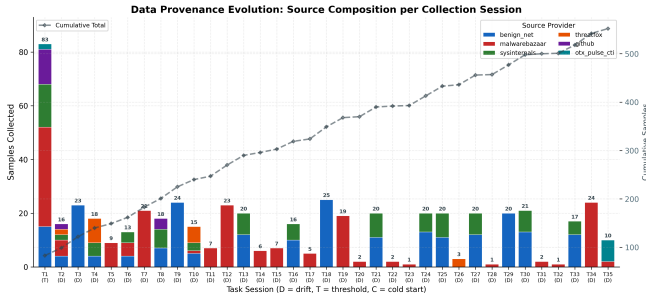


Fig. 4. Source composition over collection sessions.

20, GitHub 19, and OTX pulse CTI 10. The provenance timeline shows that collection evolves across sessions instead of staying fixed on one source. Provider outcomes are stored in SQLite, allowing low-yield sources to be ranked lower or cooled down while the agent continues collecting from productive malware and benign providers.

D. Drift and Retraining Impact

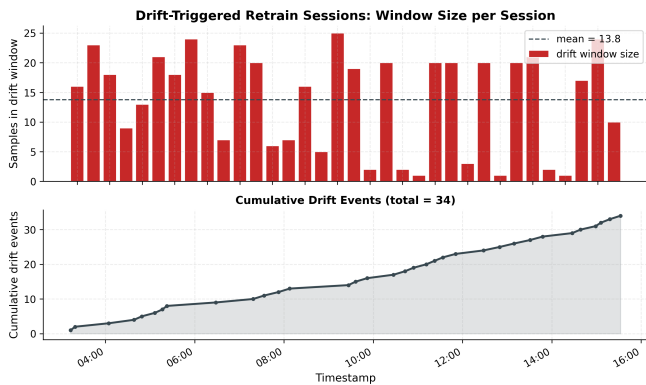


Fig. 5. Drift-triggered retrain sessions and cumulative drift events.

The system records 34 drift events, with an average drift window of 13.8 samples. Each drift cycle is documented in `drift_log.jsonl`, while model-update comparisons are written to `model_update_log.jsonl`. The pre/post retrain graph shows mixed individual outcomes, including both improvements and regressions, but the mean accuracy

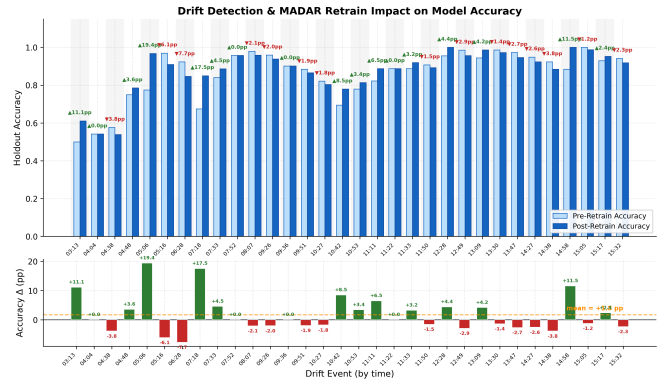


Fig. 6. Pre/post retrain accuracy delta for drift-triggered updates.

delta is positive at approximately +1.6 percentage points. This supports the use of MADAR replay as an adaptive recovery mechanism while still exposing cases where retraining does not immediately improve the strict holdout.

E. Model Performance

To establish the current operational baseline of the pipeline, Figure 7 and Table II present the evaluation metrics for the active LightGBM ml model. The model achieves an exceptional overall accuracy of 98.69% and an Area Under the Curve (AUC) of 0.9989.

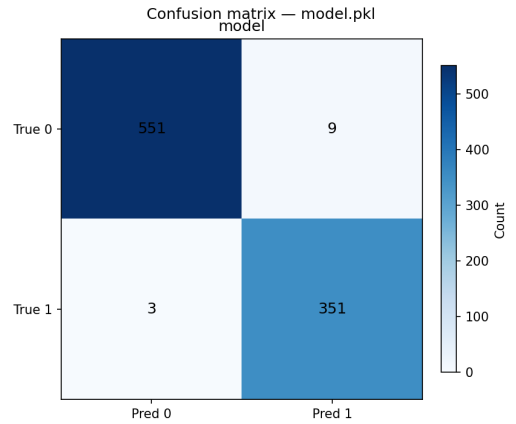


Fig. 7. Confusion matrix of the active production model.

TABLE II
ACTIVE PRODUCTION MODEL METRICS

Accuracy	Precision	Recall (TPR)	FPR	AUC
98.69%	97.50%	99.15%	1.61%	0.9989

The dynamic threshold calibration successfully bounds the False Positive Rate to 1.61% while maintaining a highly aggressive True Positive Rate (Recall) of 99.15%. As visualized in the confusion matrix, this strict recall-favored tuning results in only 3 False Negatives out of the testing

sample, ensuring that novel malware variants do not easily bypass the static feature filters during the steady state.

*The evaluation have been tested on data set of 914.

VI. CRITICAL ANALYSIS AND LIMITATIONS

- The agent successfully avoids a predefined dataset by collecting from live providers, but this makes throughput dependent on MalwareBazaar, OTX, ThreatFox, GitHub, and benign repositories. Provider cooldowns reduce wasted attempts, but rate limits and downtime still affect collection.
- The results show that MADAR replay usually helps recovery, but not every retraining event improves the strict holdout. This is why the system logs before/after deltas instead of assuming each update is beneficial.
- Drift detection is intentionally disabled during bootstrap, so early sparse labels can bypass drift validation until the initial corpus and model are ready.
- TESSERACT evaluation is more realistic than random splits, but it must skip windows where the chronological holdout is too small or single-class.
- Static PE features are safe and fast, but packed or obfuscated binaries can hide imports, strings, and opcode patterns, increasing false-negative risk.
- Ollama improves CTI filtering and drift explanations, but remains best-effort. Deterministic validation and fallback logic are still required for safety.

VII. CONCLUSION

AMD-Agent demonstrates a Docker-first autonomous malware detection agent that builds its dataset from live sources rather than a predefined corpus. The system combines LangGraph orchestration, safe PE acquisition, static feature extraction, LLM-assisted CTI filtering, provider reliability tracking, ADWIN drift detection, MADAR replay retraining, and strict chronological before/after evaluation. The results show that the agent can continue collecting data, detect drift, and usually recover performance after retraining, while still exposing regressions through model-update logs. Future work includes expanding LangGraph subgraphs for multi-agent validation, improving source discovery, and integrating reinforcement learning for source selection.

REFERENCES

- [1] Ifraimov, Baruh and Cohen, Dor, “Amd-agent,” 2026. [Online]. Available: <https://github.com/baruhifraimov/AMD-Agent>
- [2] abuse.ch, “Malwarebazaar,” 2026. [Online]. Available: <https://bazaar.abuse.ch/>
- [3] E. Carrera, “pefile: Python pe parsing module,” 2026. [Online]. Available: <https://github.com/erocarrera/pefile>
- [4] H. S. Anderson and P. Roth, “EMBER: An open dataset for training static pe malware machine learning models,” *arXiv preprint arXiv:1804.04637*, 2018. [Online]. Available: <https://arxiv.org/abs/1804.04637>
- [5] River Developers, “Adwin - river documentation,” 2026. [Online]. Available: <https://riverml.xyz/latest/api/drift/ADWIN/>
- [6] M. S. Rahman, S. E. Coull, Q. Yu, and M. Wright, “MADAR: Efficient continual learning for malware analysis with diversity-aware replay,” *arXiv preprint arXiv:2502.05760*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.05760>

- [7] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631. [Online]. Available: <https://dl.acm.org/doi/10.1145/3292500.3330701>
- [8] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, “TESSERACT: Eliminating experimental bias in malware classification across space and time,” in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, 2019, pp. 729–746. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury>
- [9] LangChain, “Langgraph documentation,” 2026. [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [10] Ollama, “Ollama documentation,” 2026. [Online]. Available: <https://ollama.com/>
- [11] AlienVault, “Open threat exchange api,” 2026. [Online]. Available: <https://otx.alienvault.com/api>